Chapter 2

Performing Complex Classifications

IN THIS CHAPTER

» Discovering publicly available image datasets

» Augmenting images in various ways

» Delving into traffic sign recognition

Deep neural network solutions have been highly successful in the image-recognition field. The great part of this technology's success, especially in AI applications, comes from three key characteristics: the availability of suitable data to train and test image networks; the application of deep neural networks to different problems thanks to transfer learning; and increasing sophistication of the technology, which allows it to answer complex questions about image content.

In this chapter, you delve into the topic of object classification and detection challenges to discover their contribution to the foundation of the present deep learning renaissance. Competitions, such as those based on the ImageNet dataset, provide the right data to train reusable networks for different purposes (thanks to transfer learning, as discussed in Book 4, Chapter 3). Competitions serve another purpose as well, which is to push researchers to find new and smarter solutions for increasing the a neural network's capability to understand images.

The chapter closes with an example of how to use an image dataset. Using the dataset, you build your own CNN for recognizing traffic signs using image augmentation and weighting for balancing the frequency of different classes in the examples.

REMEMBER You don't have to type the source code for this chapter manually. In fact, using the downloadable source is a lot easier. The source code for this chapter appears in the `DSPD_0502_Classification.ipynb` source code file for Python and the `DSPD_R_0502_Classification.ipynb` source

code file for R. See the Introduction for details on how to find these source files.

Using Image Classification Challenges

The CNN layers for image recognition were first conceived by Yann LeCun and a team of researchers. AT&T actually implemented LeNet5 (the neural network for handwritten numbers described in Book 4, Chapter 3) into ATM check readers. However, the invention didn't prevent another AI winter, starting in the 1990s, with many researchers and investors losing faith again that computers could achieve any progress toward having a meaningful conversation with humans, translating from different languages, understanding images, and reasoning in the manner of human beings.

Actually, expert systems had already undermined public confidence. *Expert systems* are a set of automatic rules set by humans to allow computers to perform certain operations. Nevertheless, the new AI winter prevented neural networks from being developed in favor of different kinds of machine learning algorithms. At the time, computers lacked computational power and had certain limits, such as the vanishing gradient problem. (Book 4, Chapter 2 discusses the vanishing gradient and other limitations that prevented deep neural architectures.) The data also lacked complexity at the time, and consequently a complex and revolutionary CNN like LeNet5, which already worked with the technology and limitations of the time, had little opportunity to show its true power.

Only a handful of researchers, such as Geoffrey Hinton, Yann LeCun, Jürgen Schmidhuber, and Yoshua Bengio, kept developing neural network technologies striving to get a breakthrough that would have ended the AI winter. Meanwhile, 2006 saw an effort by Fei-Fei Li, a computer science professor at the University of Illinois Urbana-Champaign (now an associate professor at Stanford, as well as the director of the Stanford Artificial Intelligence Lab and the Stanford Vision Lab) to provide more real-world datasets to better test algorithms. She started amassing an incredible number of images, representing a large number of object classes. You can read about this effort in the "Unveiling successful architectures" section of Book 4, Chapter 3. The proposed classes range through different types of objects, both natural (for instance, 120 dog breeds) and human made (such as means of transportation). You can explore them all at `http://image-net.org/challenges/LSVRC/2014/browse-synsets`. By using this huge image dataset for training, researchers noticed that their algorithms started working better (nothing like ImageNet existed at that time) and then they started testing new ideas and improving neural network architectures.

OBTAINING A HUMAN PERSPECTIVE IN IMAGE CLASSIFICATION

Convolutional Neural Networks (CNNs) have seen considerable use in image recognition, and you can find CNNs discussed in Book 4, Chapter 3. However, it's possible to extend CNNs using other technologies, such as those described in this chapter. Local response normalization and inception modules are technological solutions that are too complex to discuss in this book, but you should be aware that they're revolutionary. All were introduced by neural networks that won the ImageNet competition: AlexNet (in 2012); GoogleLeNet (in 2014); and ResNet (in 2015). You can read more about this technology at `https://towardsdatascience.com/difference-between-local-response-normalization-and-batch-normalization-272308c034ac` and `https://prateekvjoshi.com/2016/04/05/what-is-local-response-normalization-in-convolutional-neural-networks/`.

Delving into ImageNet and Coco

The impact and importance of the ImageNet competition (also known as ImageNet Large Scale Visual Recognition Challenge (ILSVRC; `http://image-net.org/challenges/LSVRC/`) on the development of deep learning solutions for image recognition can be summarized in three key points:

- **Helping establish a deep neural network renaissance:** The AlexNet CNN architecture (developed by Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton) won the 2012 ILSVRC challenge by a large margin over other solutions.
- **Pushing various teams of researchers to develop more sophisticated solutions:** ILSVRC advanced the performance of CNNs. VGG16, VGG19, ResNet50, Inception V3, Xception, and NASNet are all neural networks tested on ImageNet images that you can find in the Keras package (`https://keras.io/applications/`). Each architecture represents an improvement over the previous architectures and introduces key deep learning innovations.
- **Making transfer learning possible:** The ImageNet competition helped make available the set of weights that made them work. The 1.2 million ImageNet training images, distributed over 1,000 separate classes, helped create convolutional networks whose upper layers can actually generalize to problems other than ImageNet.

Recently, a few researchers started suspecting that the more recent neural architectures are overfitting the ImageNet dataset. After all, the same test set has been used for many years to select the best networks, as researchers Benjamin Recht, Rebecca Roelofs, Ludwig Schmidt, and Vaishaal Shankar speculate at `https://arxiv.org/pdf/1806.00451.pdf`.

REMEMBER Other researchers from the Google Brain team (Simon Kornblith, Jonathon Shlens, and Quoc V. Le) have discovered a correlation between the accuracy obtained on ImageNet and the performance obtained by transfer learning of the same network on other datasets. They published their findings in the paper "Do Better ImageNet Models Transfer Better?" (`https://arxiv.org/pdf/1805.08974.pdf`). Interestingly, they also pointed out that if a network is overtuned on ImageNet, it could experience problems generalizing. It is therefore a good practice to test transfer learning based on the most recent and best performing network found on ImageNet, but not to stop there. You may find that some less performing networks are actually better for your problem.

Other objections about using ImageNet is that common pictures in everyday scenes contain more objects and that these objects may not be clearly visible when partially obstructed by other objects or because they mix with the background. If you want to use an ImageNet pretrained network in an everyday context, such as when creating an application or a robot, the performance may disappoint you. Consequently, since the ImageNet competition stopped (organizers claimed that improving performance by continuing to work on the dataset wouldn't be possible), researchers have increasingly focused on using alternative public datasets to challenge one's own CNNs and improve the state of the art in image recognition. Here are the alternatives so far:

- **PASCAL VOC (Visual Object Classes)** `http://host.robots.ox.ac.uk/pascal/VOC/`**:** Developed by the University of Oxford, this dataset sets a neural network training standard for labeling multiple objects in the same picture, the PASCAL VOC xml standard. The competition associated with this dataset was halted in 2012.
- **SUN** `https://groups.csail.mit.edu/vision/SUN/`**:** Created by the Massachusetts Institute of technology (MIT), this dataset provides benchmarks to help you determine your CNN performance. No competition is associated with it.
- **MS COCO** `http://cocodataset.org/`**:** Prepared by Microsoft Corporation, this dataset offers a series of active competitions.

In particular, the Microsoft Common Objects in the Context dataset (hence the name MS COCO) offers fewer training images for your model than you find in ImageNet, but each image contains multiple objects. In addition, all objects appear in realistic positions (not staged) and settings (often in the open air and in public settings such as roads and streets). To distinguish the objects, the dataset provides both contours in pixel coordinates and labeling in the PASCAL VOC XML standard, having each object defined not just by a class but also by its coordinates in the images (a picture rectangle that shows where to find it). This rectangle is called a

bounding box, defined in a simple way using four pixels, in contrast to the many pixels necessary for defining an object by its contours.

REMEMBER The ImageNet dataset has recently started offering, in at least one million images, multiple objects to detect as well as their bounding boxes.

Learning the magic of data augmentation

Even if you have access to large amounts of data for your deep learning model, such as the ImageNet and MS COCO datasets, that may be not enough because of the multitude of parameters found in most complex neural architectures. In fact, even if you use techniques such as dropout (as explained in the "Adding regularization by dropout" section of Book 4, Chapter 2), overfitting is still possible. *Overfitting* occurs when the network memorizes the input data and learns no generally useful data patterns. Apart from dropout, other techniques that could help a network fight overfitting are LASSO, Ridge, and ElasticNet. However, nothing is as effective for enhancing your neural network's predictive capabilities as adding more examples to your training schedule.

REMEMBER Originally, LASSO, Ridge, and ElasticNet were ways to constrain the weights of a linear regression model, which is a statistical algorithm for computing regression estimates. In a neural network, they work in a similar way by forcing the total sum of the weights in a network to be the lowest possible without harming the correctness of predictions. LASSO strives to put many weights down to zero, thus achieving a selection of the best weights. By contrast, Ridge instead tends to dampen all the weights, avoiding higher weights that can generate overfitting. Finally, ElasticNet is a mix of the LASSO and Ridge approaches, amounting to a trade-off between the selection and dampening strategies.

Image augmentation provides a solution to the problem of a lack of examples to feed a neural network to artificially create new images from existing ones. *Image augmentation* consists of different image-processing operations that are carried out separately or conjointly to produce an image different from the initial one. The result helps the neural network learn its recognition task better.

For instance, if you have training images that are too bright or too blurry, image processing modifies the existing images into darker and sharper

versions. These new versions exemplify the characteristics that the neural network must focus on, rather than provide examples that focus on image quality. In addition, turning, cutting, or bending the image, as shown in Figure 2-1, could help because, again, they force the network to learn useful image features, no matter how the object appears.

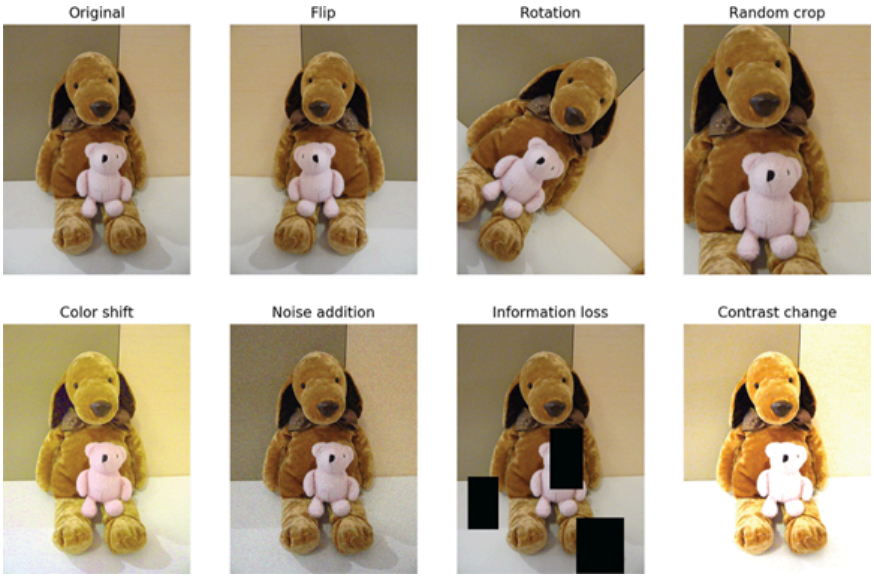

FIGURE 2-1: Some common image augmentations.

The most common image augmentation procedures, as shown in Figure 2-1, are

- **Flip:** Flipping your image on its axis tests the algorithm's capability to find it regardless of perspective. The overall sense of your image should hold even when flipped. Some algorithms can't find objects when upside down or even mirrored, especially if the original contains words or other specific signs.
- **Rotation:** Rotating your image allows algorithm testing at certain angles, simulating different perspectives or imprecisely calibrated visuals.
- **Random crop:** Cropping your image forces the algorithm to focus on an image component. Cutting an area and expanding it to the same size of a standard image enables you to test for recognition of partially hidden image features.
- **Color shift:** Changing the nuances of image colors generalizes your example because the colors can change or be recorded differently in the real world.
- **Noise addition:** Adding random noise tests the algorithm's capability to detect an object even when object quality is less than perfect.
- **Information loss:** Randomly removing parts of an image simulates visual obstruction. It also helps the neural network rely on general image features, not on particulars (which could be randomly eliminated).
- **Contrast change:** Changing the luminosity makes the neural network less sensitive to the light conditions (for instance, to daylight or artificial light).

TIP You don't need to specialize in image processing to leverage this powerful image-augmentation technique. Keras offers a way to easily in-

corporate augmentation into any training using the `ImageDataGenerator` function (`https://faroit.github.io/keras-docs/1.2.2/preprocessing/image/`).

The `ImageDataGenerator`'s main purpose is to generate batches of inputs to feed your neural network. This means that you can get your data as chunks from a NumPy array using the `.flow` method. In addition, you don't need to have all the training data in memory because the `.flow_from_directory` method can get it for you directly from disk. As `ImageDataGenerator` pulls the batches of images, it can transform them using rescaling (images are made of integers, ranging from 0 to 255, but neural networks work best with floats ranging from zero to one) or by applying some transformations, such as

- **Standardization:** Getting all your data on the same scale by setting the mean to zero and the standard deviation to one (as the statistical standardization), based on the mean and standard deviation of the entire dataset (*feature-wise*) or separately for each image (*sample-wise*).
- **ZCA whitening:** Removing any redundant information from the image while maintaining the original image resemblance.
- **Random rotation, random shifts, and random flips:** Orienting, shifting, and flipping the image so that objects appear in a different pose than the original.
- **Reordering dimensions:** Matching the dimensions of data between images. For instance, converting BGR images (a color image format previously popular among camera manufacturers) into standard RGB.

When you use `ImageDataGenerator` to process batches of images, you're not bound by the size of computer memory on your system, but rather by your storage size (for instance, the size of your hard disk) and its speed of transfer. You could even get the data you need on the fly from the Internet, if your connection is fast enough.

TIP You can get even more powerful image augmentations using a package such as the albumentations package (`https://github.com/albu/albumentations`). Alexander Buslaev, Alex Parinov, Vladimir I. Iglovikov, and Evegene Khvedchenya created it based on their experience with many image-detection challenges. The package offers an incredible array of possible image processing tools based on the task to accomplish and the kind of neural network you use.

Distinguishing Traffic Signs

After discussing the theoretical grounds and characteristics of CNNs, you can try building one. TensorFlow and Keras can construct an image clas-

sifier for a specific delimited problem. Specific problems don't imply learning a large variety of image features to accomplish the task successfully. Therefore, you can easily solve them using simple architectures, such as LeNet5 (the CNN that revolutionized neural image recognition, discussed in Book 4, Chapter 3) or something similar. This example performs an interesting, realistic task using the German Traffic Sign Recognition Benchmark (GTSRB) found at this Institute für NeuroInformatik at Ruhr-Universität Bochum page: `http://benchmark.ini.rub.de/?section=gtsrb&subsection=about`.

REMEMBER Reading traffic signs is a challenging task because of differences in visual appearance in real-world settings. The GTSRB provides a benchmark to evaluate different machine learning algorithms applied to the task. You can read about the construction of this database in the paper by J. Stallkamp and others called "Man vs. computer: Benchmarking machine learning algorithms for traffic sign recognition" at `https://www.ini.rub.de/upload/file/1470692859_c57fac98ca9d02ac701c/stallkampetal_gtsrb_nn_s`

The GTSRB dataset offers more than 50,000 images arranged in 42 classes (traffic signs), which allows you to create a multiclass classification problem. In a multiclass classification problem, you state the probability of the image's being part of a class and take the highest probability as the correct answer. For instance, an "Attention: Construction Site" sign will cause the classification algorithm to generate high probabilities for all attention signs. (The highest probability should match its class.) Blurriness, image resolution, different lighting, and perspective conditions make the task challenging for a computer (as well as sometimes for a human), as you can see from some of the examples extracted from the dataset in Figure 2-2.

FIGURE 2-2: Some examples from the German Traffic Sign Recognition Benchmark.

Preparing the image data

The example begins by configuring the model, setting the optimizer, preprocessing the images, and creating the convolutions, the pooling, and the dense layers, as shown in the following code. (See Book 1, Chapter 5 for how to work with TensorFlow and Keras.)

```
import numpy as np
import zipfile
import pprint

from skimage.transform import resize
from skimage.io import imread

import matplotlib.pyplot as plt
%matplotlib inline

import warnings
warnings.filterwarnings("ignore")
```

Here are the Keras-specific imports needed for this example:

```
from keras.models import Sequential
from keras.optimizers import Adam
from keras.preprocessing.image import ImageDataGenerator
from keras.utils import to_categorical
from keras.layers import Conv2D, MaxPooling2D
from keras.layers import (Flatten, Dense, Dropout)
```

TIP The dataset comprises more than 50,000 images, and the associated neural network can achieve a near-human level of accuracy in recognizing traffic signs. Such an application will require a large amount of

computer calculations, and running this code locally could take a long time on your computer, depending on the kind of computer you have. Likewise, Colab can take longer depending on the resources that Google makes available to you, including whether you actually have access to a GPU, as mentioned in Book 1, Chapter 3. Timing this initial application on your setup will help you know whether your local machine or Colab is the fastest environment in which to run larger datasets. However, the best environment is the one that produces the most consistent and reliable results. You may not have a solid Internet connection to use, making Colab a poorer choice.

At this point, the example retrieves the GTSRB dataset from its location on the Internet (the INI Benchmark website, at the Ruhr-Universität Bochum specified previously). The following code snippet downloads it to the same directory as the Python code. Note that the download process can take a little time to complete, so now might be a good time to refill your teacup.

```
import urllib.request
import os.path
if not os.path.exists("GTSRB_Final_Training_Images.zip"):
    url = "https://sid.erda.dk/public/archives/\
    daaeac0d7ce1152aea9b61d9f1e19370/\
    GTSRB_Final_Training_Images.zip"
    filename = "./GTSRB_Final_Training_Images.zip"
    urllib.request.urlretrieve(url, filename)
```

After retrieving the dataset as a `.zip` file from the Internet, the code sets an image size. (All images are resized to square images, so the size represents the sides in pixels.) The code also sets the portion of data to keep for testing purposes, which means excluding certain images from training to have a more reliable measure of how the neural network works.

```
IMG_SIZE = 32
TEST_SIZE = 0.2
```

A loop through the files stored in the downloaded `.zip` file retrieves individual images, resizes them, stores the class labels, and appends the images to two separate lists: one for the training and one for testing pur-

poses. The sorting uses a hash function, which translates the image name into a number and, based on that number, decides where to append the image:

```
X, Xt, y, yt = list(), list(), list(), list()

archive = zipfile.ZipFile(
          'GTSRB_Final_Training_Images.zip', 'r')
file_paths = [file for file in archive.namelist()
              if '.ppm' in file]

for filename in file_paths:
    img = imread(archive.open(filename))
    img = resize(img,
                 output_shape=(IMG_SIZE, IMG_SIZE),
                 mode='reflect')
    img_class = int(filename.split('/')[-2])

    if (hash(filename) % 1000) / 1000 > TEST_SIZE:
        X.append(img)
        y.append(img_class)
    else:
        Xt.append(img)
        yt.append(img_class)
archive.close()
```

After the job is completed, the code reports the consistency of the train and test examples:

```
test_ratio = len(Xt) / len(file_paths)

print("Train size:{} test size:{} ({:0.3f})".format(

    len(X),

    len(Xt),

    test_ratio))
```

The train size is more than 30,000 images, and the test almost is 8,000 (20 percent of the total):

```
Train size:31344 test size:7865 (0.201)
```

Your results may vary a little from those shown. For example, another run of the example produced a train size of 31,415 and a test size of 7,794. Neural networks can learn multiclass problems better when the classes are numerically similar or they tend to concentrate their attention on learning just the more populated classes. The following code checks the distribution of classes:

```
classes, dist = np.unique(y+yt, return_counts=True)

NUM_CLASSES = len(classes)

print ("No classes:{}".format(NUM_CLASSES))

plt.bar(classes, dist, align='center', alpha=0.5)

plt.show()
```

Figure 2-3 shows that the classes aren't balanced. Some traffic signs appear more frequently than others do (for instance, while driving, stop signs are encountered more frequently than a deer crossing sign).

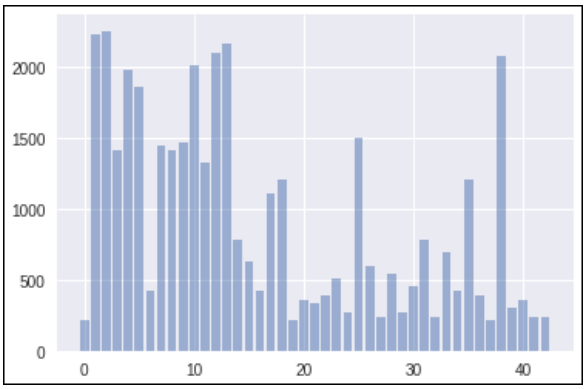

FIGURE 2-3: Distribution of classes.

As a solution, the code computes a *weight*, which is a ratio based on frequencies of classes that the neural network uses to increase the signal it receives from rarer examples and to dump the more frequent ones:

```
class_weight = {c:dist[c]/np.sum(dist) for c in classes}
```

Running a classification task

After setting the weights, the code defines the image generator, the part of the code that retrieves the images in batches (samples of a predefined size) for training and validation, normalizes their values, and applies augmentation to fight overfitting by slightly shifting and rotating them. Notice that the following code applies augmentation only on the training image generator, not the validation generator, because it's necessary to test the original images only.

```
batch_size = 256

tgen=ImageDataGenerator(rescale=1./255,
                        rotation_range=5,
                        width_shift_range=0.10,
                        height_shift_range=0.10)

train_gen = tgen.flow(np.array(X),
                      to_categorical(y),
                      batch_size=batch_size)
```

Here is the code for the validation generator:

```
vgen=ImageDataGenerator(rescale=1./255)

val_gen = vgen.flow(np.array(Xt),
                    to_categorical(yt),
                    batch_size=batch_size)
```

The code finally builds the neural network:

```
def small_cnn():
    model = Sequential()
    model.add(Conv2D(32, (5, 5), padding='same',
                     input_shape=(IMG_SIZE, IMG_SIZE, 3),
                     activation='relu'))
    model.add(Conv2D(64, (5, 5), activation='relu'))
    model.add(Flatten())
    model.add(Dense(768, activation='relu'))
    model.add(Dropout(0.4))
    model.add(Dense(NUM_CLASSES, activation='softmax'))
    return model


model = small_cnn()
model.compile(loss='categorical_crossentropy',
              optimizer=Adam(),
              metrics=['accuracy'])
```

The neural network consists of two convolutions, one with 32 channels, the other with 64, both working with a kernel of size (5,5). The convolutions are followed by a dense layer of 768 nodes. Dropout (dropping 40 percent of the nodes) regularizes this last layer, and softmax activates it (thus the sum of the output probabilities of all classes will sum to 100 percent).

On the optimization side, the loss to minimize is the categorical cross-entropy. The code measures success on *accuracy*, which is the percentage of correct answers provided by the algorithm. (The traffic sign class with the highest predicted probability is the answer.)

```
history = model.fit_generator(
    train_gen,
    steps_per_epoch=len(X) // batch_size,
    validation_data=val_gen,
    validation_steps=len(Xt) // batch_size,
    class_weight=class_weight,
    epochs=100,
    verbose=2)
```

Using the `fit_generator` on the model, the batches of images start being randomly extracted, normalized, and augmented for the training phase. After pulling out all the training images, the code sees an *epoch* (a training iteration using a full pass on the dataset) and computes a validation score on the validation images. After reading 100 epochs, the training and the model are completed.

TIP If you don't use any augmentation, you can train your model in just about 30 epochs and reach a performance of your model that is almost comparable to a driver's skill in recognizing the different kinds of traffic signs (which is about 98.8 percent accuracy). The more aggressive the augmentation you use, the more epochs necessary for the model to reach its top potential, although accuracy performances will be higher, too. Here is the code that outputs the validation accuracy:

```
print("Best validation accuracy: {:0.3f}"
      .format(np.max(history.history['val_acc'])))
```

CONSIDERING THE COST OF REALISTIC OUTPUT

As mentioned a few times in this book already, deep learning training can take a considerable amount of time to complete. Whenever you see a fit function in the code, such as `model.fit_generator`, you're likely asking the system to perform training. The example code will always strive to provide you with realistic output — that is, what a scientist in the real world would consider acceptable.

Unfortunately, realistic output may cost you too much in the way of time. Not everyone has access to the latest high-technology system, and not everyone will get a GPU on Colab. The example in this chapter consumes a great deal of time to train in some cases. For example, testing the code on Colab took a little over 16 hours to complete when Colab didn't provide a GPU. The same example might run in as little as an hour if Colab does provide a GPU. (Book 1, Chapter 3 tells you more about the GPU issue.) Likewise, using a CPU-only system, a 16-core Xeon system required 4 hours and 23 minutes to complete training, but an Intel i7 processor with 8 cores took a little over 9 hours to do the same thing.

One way around this issue is to change the number of epochs used to train your model. The `epochs=100` setting used for the example in this chapter provides an output accuracy of a little over 99 percent. However, if time is a factor, you may want to use a lower `epochs` setting when running this example to reduce the time you wait for the example to complete.

Another alternative for avoiding the problem is using GPU support on your local machine. However, to use this alternative, you must have a display adapter with the right kind of chip. Because the setup is complex and you're not likely to have the right GPU, this book takes the CPU-only route. However, you can certainly install the correct support by using Book 1, Chapter 3 as a starting point and then adding CUDA support. The article at `https://towardsdatascience.com/tensorflow-gpu-installation-made-easy-use-conda-instead-of-pip-52e5249374bc` provides additional details.

At this point, the code plots a graph depicting how the training and validation accuracy behaved during training:

```
plt.plot(history.history['acc'])
```

```
plt.plot(history.history['val_acc'])
```

```
plt.ylabel('accuracy'); plt.xlabel('epochs')
```

```
plt.legend(['train', 'test'], loc='lower right')
```

```
plt.show()
```

The code will report to you the best validation accuracy recorded and plot the accuracy curves achieved on training and validation data during the increasing epochs of learning, as shown in Figure 2-4. Notice how the training and validation accuracies are nearly similar at the end of training, although the validation is always better than the training. That's eas-

ily explained because the validation images are actually “easier” to guess than the training images because no augmentation is applied to them.

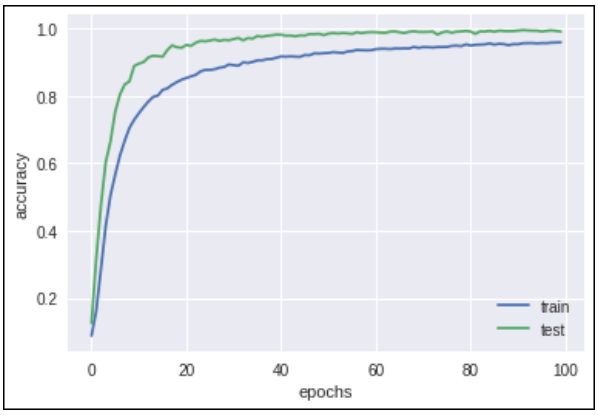

FIGURE 2-4: Training and validation errors compared.

Given that the code can initialize the neural network in different ways, you may see different best results at the end of the training optimization. However, by the end of the 100 epochs set in the code, the validation accuracy should exceed 99 percent. (Sample runs achieved up to 99.5 percent on Colab.)

REMEMBER A difference exists between the performance you obtain on the train data (which is often less) and on your validation subset, because train data is more complex and variable than validation data, given the image augmentations that the code sets up.

TECHNICAL STUFF You should consider this result to be quite an excellent one based on the state-of-the-art benchmarks that you can read about in the paper called “HALOI, Mrinal. Traffic sign classification using deep inception-based convolutional networks” (`https://arxiv.org/pdf/1511.02992.pdf`). The paper hints at what can be easily achieved in terms of image recognition on limited problems using clean data and readily available tools such as TensorFlow and Keras.

Table of contents collapsed